4 串

4.1 串、子串与基本运算

串是零个或多个字符组成的有限序列，长度为字符数。空串长度 0，空格串含实际空格字符。子串由主串中连续字符构成，位置取其首字符位置；相同子串可有多个出现位置。

两串相等要求长度相同且对应字符相同。字典序比较第一对不同字符；已比较部分相同且一方先结束时，较短串较小。

操作	结果与条件
赋值、复制	容量足够；复制地址与独立复制字符的语义须明确
判空、求长	依赖长度成员或结束标志
清空、销毁	清空保留存储；销毁回收动态存储
连接	总长度可容纳
取子串	检查起点、长度和末位置
比较	返回正、零、负值表示大小
定位	本章首次成功返回 1 起始位置，失败返回 0

允许空子串时，长 n 的串须满足 $1 \le pos \le n+1$、$len \ge 0$、$pos+len-1 \le n$；`pos=n+1` 只允许 `len=0`。模式匹配约定空模式返回 1。

4.2 存储表示与基本操作代码

本章顺序串的字符从 1 开始，`ch[0]`弃用，长度另存。`ch[MAXLEN]`的有效字符容量为 `MAXLEN-1`，保持 $0 \le length < MAXLEN$。

```c
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN];
    int length;
} SString;
```

显式长度表示无需`'\0'`结束符；转用 C 字符串库前须转换布局。其他布局如首单元保存长度、0 起始另存长度、结束符表示等，均有各自容量与求长规则。

```
bool SubString(SString &Sub, SString S, int pos, int len) {
    if (pos < 1 || pos > S.length + 1 || len < 0 ||
        len > S.length - pos + 1)
        return false;
    for (int i = pos; i < pos + len; ++i)
        Sub.ch[i - pos + 1] = S.ch[i];
    Sub.length = len;
    return true;
}
int StrCompare(SString S, SString T) {
    for (int i = 1; i <= S.length && i <= T.length; ++i)
        if (S.ch[i] != T.ch[i])
            return (unsigned char)S.ch[i] -
                   (unsigned char)T.ch[i];
    return S.length - T.length;
}
```

```
bool Concat(SString &T, SString S1, SString S2) {
    if (S1.length + S2.length >= MAXLEN) return false;
    for (int i = 1; i <= S1.length; ++i)
        T.ch[i] = S1.ch[i];
    for (int i = 1; i <= S2.length; ++i)
        T.ch[S1.length + i] = S2.ch[i];
    T.length = S1.length + S2.length;
    return true;
}
```

`SubString` 时间 $O(len)$；比较最坏 $O(\min(n,m))$；连接时间 $O(n+m)$。上述定长串按值传入副本，输出与输入同名时仍能读取原值。`unsigned char` 使字符编码按无符号值比较。

堆串采用 `char *ch; int length;`管理动态连续区，需处理分配、复制、释放。链串可每结点保存一字符或多字符形成块，块尾填充须与真实字符区分。仅计字符与指针时，每结点 q 个 1 字节字符、指针 p 字节的存储密度为 $q/(q+p)$；结构体对齐另计。

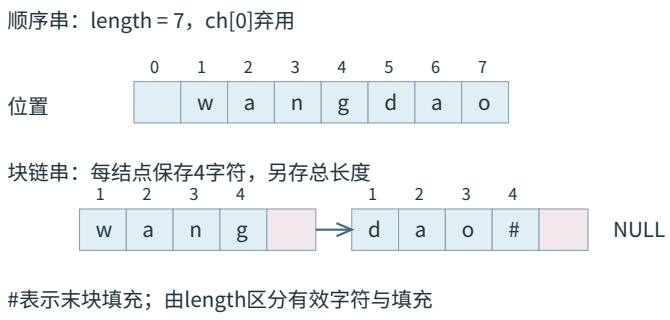


Figure 4.1: 顺序串与块链串

4.3 朴素模式匹配

主串长 n、模式长 m，字符位置均从 1 开始。失配时本轮起点为 $i-j+1$，改从下一起点比较：`i=i-j+2; j=1`。

```
int Index(SString S, SString T) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) {
            ++i; ++j;
        } else {
            i = i - j + 2;
            j = 1;
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

成功时 $j = m + 1$，返回 $i - m$；主串先耗尽则失败。非空模式最多有 $n - m + 1$ 个候选起点，最坏时间 $O((n - m + 1)m)$，常记 $O(nm)$；辅助工作空间 $O(1)$。首起点成功时比较 m 次。

4.4 KMP 与 next 的 1 起始约定

在 (i, j) 失配前，$S[i - j + 1..i - 1] = T[1..j - 1]$。模式移位后仍保留的重合部分，必须是该已匹配前缀的相等真前后缀。

设 ℓ 为串 $T[1..j - 1]$ 的最长相等真前后缀长度，则

$$next[1] = 0, \qquad next[j] = \ell + 1 \quad (j > 1).$$

`next[j]`是下一次比较的模式位置。失配令 `j=next[j]`，主串位置保持；`j==0` 时下一轮同时推进 `i、j`，恢复模式首字符。

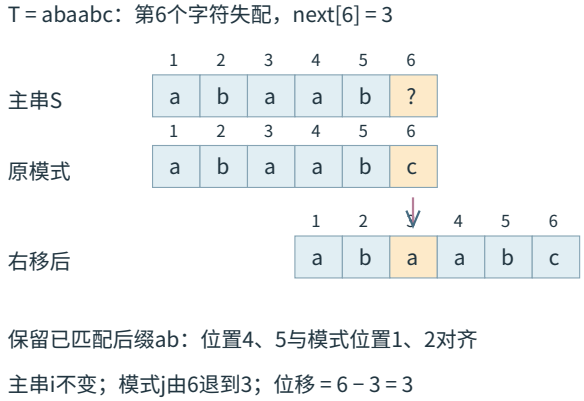


Figure 4.2: 失配后保留已匹配段的边界

```
int Index_KMP(SString S, SString T, int next[]) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) {
            ++i;
            ++j;
        } else {
            j = next[j];
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

前缀函数换算

本章 next 值对应当前位置之前的前缀。若另有 0 起始前缀函数 π，则 $j>1$ 时 $next[j]=\pi[j-2]+1$；换定义时匹配代码也须同步。

短路求值保证 `j==0` 时不读取 `T.ch[0]`。主串位置单调递增，同一个主串字符可能与多个模式位置比较。

例 4-1

主串 `S=abaabaabcabaabc`，模式 `T=abaabc`。分别用朴素匹配和普通 next 表的 KMP 查找首次出现位置，计算实际字符比较次数。字符位置与 next 均从 1 开始，`next[1]=0`。

线性时间建立 next

已知 `next[i]=j`：若 `j==0` 或 `T.ch[i]==T.ch[j]`，则下一前缀的边界可扩展，写入 `next[++i]=++j`；失配时沿 `next[j]`尝试更短边界。

```
void get_next(SString T, int next[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    next[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            next[i] = j;
        } else {
            j = next[j];
        }
    }
}
```

模式	普通 next
`google`	`0,1,1,1,2,1`
`abaabc`	`0,1,1,2,2,3`
`ababaa`	`0,1,1,2,3,4`
`aaaab`	`0,1,2,3,4`

`google` 在位置 5 之前的 `goog` 有长度 1 的边界 `g`，故 `next[5]=2`。主串 `googlogoogoogle` 首次匹配 `google` 的位置为 10。

4.5 nextval 优化

已知 `S.ch[i]`与 `T.ch[j]`失配, 若 `T.ch[next[j]]==T.ch[j]`, 退到 `next[j]`后仍必败。令 `nextval[1]=0`, $j \geq 2$ 时:

$$nextval[j] = \begin{cases} next[j], & T[j] \neq T[next[j]], \\ nextval[next[j]], & T[j] = T[next[j]]. \end{cases}$$

```
void get_nextval(SString T, int nextval[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    nextval[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            if (T.ch[i] != T.ch[j])
                nextval[i] = j;
            else
                nextval[i] = nextval[j];
        } else {
            j = nextval[j];
        }
    }
}
```

将优化表作为 `Index_KMP` 的第三个实参，匹配循环保持相同。

模式	nextval
abaabc	0,1,0,2,1,3
ababaa	0,1,0,1,0,4
aaaab	0,0,0,0,4

例 4-2

主串 `aaacaaaabdaaaab`，模式 `aaaab`。比较普通 next 和优化 nextval 的匹配过程，求首次成功位置与两者字符比较次数。采用 1 起始位置和 `next[1]=nextval[1]=0`。

4.6 KMP 的代价与边界

建表中 i 最多推进 $m-1$ 次, j 的回退总量受此前增量限制, 建表 $O(m)$; 匹配中 i 最多推进 n 次，累积回退同样受推进约束，扫描 $O(n)$。总时间 $O(n+m)$，表空间 $O(m)$；已给表时单独扫描记为 $O(n)$。

每次跳过的对齐都由已匹配前缀的边界关系排除，故不遗漏合法起点。统计字符比较只计实际字符判等，`j==0` 检查单独计控制操作。查找全部出现时还需约定是否允许重叠，以及成功后的继续位置。

固定数组按值传参的复制成本在教材分析中按预设容量处理；可增长规模的实现应把实际复制成本计入。